I have a Python Pygame project called **Rhythm Tap** for a vibe coding assignment. I have an existing GitHub repository with a working basic rhythm game.

The assignment requires me to implement these four features:

1. Sound effects on hit — different sounds for PERFECT, GREAT and OK.
2. Hold notes — a note should require the player to hold the corresponding key for 1 second.

Show more

Absolutely. 👍

Send/upload your existing **Rhythm Tap** project files (especially the main `.py` file, plus any other Python files/assets/config files that are relevant).

I'll first **analyze your existing code without changing it** and explain:

1. **Project structure** — what each file does.
2. **Game flow** — what happens from launch → note spawning → key press → scoring → game over.
3. **Main game loop** — how Pygame updates frames and handles events.
4. **Note system** — how notes are created, stored, moved, and removed.
5. **Input/scoring system** — how the current hit detection and PERFECT/GREAT/OK/MISS logic works.
6. **Timing system** — how notes currently spawn and why it is/isn't BPM-based.
7. **Game-over logic** — what currently happens when the game ends.
8. **Exactly where each of the 4 assignment features should be added**, without unnecessarily rewriting your existing project.

Then, once you understand the architecture, we can implement the features **one at a time**, so you can explain the changes in your viva rather than just having AI generate an entire project.

Upload the files whenever you're ready.

Here are my current `main.py`, `game/game_engine.py`, and `game/beat.py`.

Please inspect them carefully and explain:

- how the game loop works
- how notes are currently spawned
- how keyboard input is handled
- how hits and misses are detected
- how PERFECT/GREAT/OK are currently calculated
- which classes and functions are responsible for each

Show more

Yes — your current project is already structured reasonably well, so you **do not need to rewrite it**. The game logic is mainly concentrated in `GameEngine`, while `Note` is a simple data/behavior class.

1. Overall architecture

You currently have three files:

```
main.py
   ↓
GameEngine
   ↓
Note objects
```

`main.py`

Very small entry point:

Python

```python
engine = GameEngine()
engine.run()
```

Its only job is to create the game engine and start the game.

`game/game_engine.py`

This is the **main controller**.

`GameEngine` is responsible for:

- initializing Pygame
- creating the window
- maintaining game state
- spawning notes
- processing keyboard input
- detecting hits
- calculating PERFECT/GREAT/OK
- detecting automatic misses
- updating notes
- drawing everything
- determining game over

Most of your four new features will therefore be implemented here.

`game/beat.py`

Contains:

```python
class Note
```

A `Note` currently stores:

- `lane`
- `y`
- `speed`
- `hit`
- `missed`

and has two methods:

```python
update()
get_rect()
```

This is where we can later add information such as:

```python
is_hold
hold_duration
holding
hold_start_time
```

without changing the entire architecture.

---

2. How the game loop works

The central loop is in:

```python
def run(self):
    running = True
    while running:
        running = self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
```

Your game runs at:

```python
FPS = 60
```

So the intended loop is approximately **60 iterations per second**.

Each iteration has four important stages.

Step 1 — Input

```python
running = self.handle_events()
```

Pygame checks for things like:

- closing the window
- pressing D
- pressing F
- pressing J
- pressing K
- pressing R

Step 2 — Game state update

Python Run

```python
self.update()
```

This:

- increments the frame
- controls note spawning
- moves notes downward
- detects notes that have passed the hit area
- records misses
- updates feedback
- checks game-over condition

Step 3 — Drawing

Python Run

```python
self.draw()
```

This redraws the entire screen:

```
background
↓
lanes
↓
hit line
↓
notes
↓
feedback
↓
score/combo/misses
↓
game-over overlay if necessary
```

Step 4 — FPS limiting

Python Run

```python
self.clock.tick(FPS)
```

Because:

```python
FPS = 60
```

Pygame attempts to keep the game running at about 60 FPS.

This is important because your **current spawning system is based on frames**, which we'll discuss next.

---

3. How notes are currently spawned

Your `reset()` initializes:

```python
self.spawn_timer = 0
self.spawn_interval = 45
```

Every frame:

```python
self.spawn_timer += 1
```

Then:

```python
if self.spawn_timer >= self.spawn_interval:
    self.spawn_note()
    self.spawn_timer = 0
```

Therefore:

```
45 frames
    ↓
spawn note
    ↓
reset timer
    ↓
45 frames
    ↓
spawn another note
```

At 60 FPS:

```
45 frames ÷ 60 FPS = 0.75 seconds
```

So you're approximately spawning:

```
1 note every 0.75 seconds
```

or:

```
80 notes/minute
```

But this isn't really BPM synchronization.

The important problem is that the game is saying:

> "Spawn a note every 45 frames."

rather than:

> "Spawn a note on every beat according to a BPM."

Random lane selection

Inside:

Python Run

```python
def spawn_note(self):
    lane = random.randint(0, LANES - 1)
    self.notes.append(Note(lane, y=-30, speed=self.speed))
```

You have four lanes:

Python Run

```python
LANES = 4
```

and:

Python Run

```
D F J K
```

The lane is randomly selected:

```
0 → D
1 → F
2 → J
3 → K
```

Then a `Note` object is created.

For example:

Python Run

```python
Note(2, y=-30, speed=5)
```

means:

```
lane = 2 → J
starting Y = -30
speed = 5 pixels/frame
```

---

4. How notes move

The `Note` class contains:

Python Run

```python
def update(self):
    self.y += self.speed
```

Every game frame, `GameEngine.update()` does:

Python Run

```python
for note in self.notes:
    note.update()
```

So if:

Python Run

```python
speed = 5
```

the note moves:

```
5 pixels
↓
5 pixels
↓
5 pixels
↓
...
```

every frame.

The note therefore travels downward toward:

Python Run

```python
HIT_Y = HEIGHT - 80
```

Since:

Python Run

```python
HEIGHT = 640
```

we get:

```
HIT_Y = 560
```

So the hit line is at approximately:

```
Y = 560
```

---

5. How keyboard input works

This happens in:

Python Run

```python
handle_events()
```

You have:

Python Run

```python
for event in pygame.event.get():
```

Pygame gives you events that happened since the previous frame.

You check:

Python Run

```python
if event.type == pygame.KEYDOWN:
```

So your game responds when a key is **pressed down**.

Then:

Python Run

```python
for i, key in enumerate(LANE_KEYS):
    if event.key == key:
        self.process_tap(i)
```

Your keys are:

Python Run

```python
LANE_KEYS = [
    pygame.K_d,
    pygame.K_f,
    pygame.K_j,
    pygame.K_k
]
```

Therefore:

Key	Lane
D	0
F	1
J	2
K	3

For example, if the player presses `J`:

```
KEYDOWN
   ↓
J detected
   ↓
lane = 2
```

```
    ↓
process_tap(2)
```

---

6. How hits are detected

The main hit detection happens in:

Python Run

```python
process_tap(self, lane)
```

First, it searches for the closest valid note in that lane:

Python Run

```python
for note in self.notes:
    if note.lane == lane and not note.hit and not note.missed:
```

Then it calculates:

Python Run

```python
dist = abs(
    note.y + Note.HEIGHT//2 - HIT_Y
)
```

This measures the distance between the **center of the note** and the hit line.

For example:

```
Note center
     ↓
   [NOTE]
     ↓
---------------- HIT LINE
```

If the distance is small, the player timed the key press well.

The closest note is stored in:

Python Run

```python
best
```

and its distance in:

Python Run

```
best_dist
```

---

7. Current hit window

You define:

Python Run

```
HIT_WINDOW = 30
```

Then:

Python Run

```
if best and best_dist <= HIT_WINDOW:
```

Therefore, the player can successfully hit a note if its center is within **30 pixels** of the hit line.

If the closest note is outside 30 pixels:

Python Run

```
else:
    self.combo = 0
    ...
    "MISS"
```

So pressing a lane key at the wrong time causes a MISS feedback.

---

8. How PERFECT / GREAT / OK are calculated

This is currently very straightforward.

Python Run

```
if best_dist < 8:
    grade, pts = "PERFECT", 300

elif best_dist < 18:
    grade, pts = "GREAT", 200
```

```
else:
    grade, pts = "OK", 100
```

So your current grading system is based entirely on **distance from the hit line**.

PERFECT

```
distance < 8 pixels
```

→ 300 points

GREAT

```
8 ≤ distance < 18
```

→ 200 points

OK

```
18 ≤ distance ≤ 30
```

→ 100 points

MISS

```
distance > 30
```

or no valid note exists.

Important viva point

Your game currently does **not** calculate accuracy using timing in milliseconds.

It calculates accuracy indirectly using:

> pixel distance between the note's center and the hit line.

That's important when we implement BPM synchronization because we'll eventually have a much more meaningful concept of **beat timing**.

---

9. What happens after a successful hit?

When a note is hit:

Python Run

```python
best.hit = True
```

Then:

Python Run

```python
self.combo += 1
```

and:

Python Run

```python
self.max_combo = max(self.max_combo, self.combo)
```

The score is:

Python Run

```python
self.score += pts * max(1, self.combo // 5)
```

So your score has a combo multiplier.

For example, at low combo:

```
300 × 1 = 300
```

At combo 10:

```
10 // 5 = 2

300 × 2 = 600
```

The note is marked:

Python Run

```python
hit = True
```

and then `draw()` skips it:

Python Run

```python
if note.hit:
    continue
```

So it disappears visually.

---

10. How automatic misses work

There's another kind of MISS.

In `update()`:

Python Run

```python
if not note.hit and not note.missed and note.y > HIT_Y + HIT_WINDOW + Note.HEIGHT:
```

If a note travels sufficiently past the hit zone without being hit:

Python Run

```python
note.missed = True
```

Then:

Python Run

```python
self.misses += 1
self.combo = 0
```

So there are currently **two ways to get a MISS**:

1. Player presses at the wrong time

`process_tap()`:

```
press key
   ↓
no note within 30 pixels
   ↓
MISS
```

2. Player doesn't press

`update()`:

```
note passes hit zone
   ↓
not hit
   ↓
automatic MISS
```

That's an important distinction for your viva.

---

11. How notes are removed

You have:

```python
self.notes = [
    n for n in self.notes
    if not (n.hit or n.missed and n.y > HEIGHT + 10)
]
```

This means:

- hit notes are removed
- missed notes are removed once they move beyond the screen

There is a subtle Python operator-precedence point here:

```python
n.hit or n.missed and n.y > HEIGHT + 10
```

is interpreted approximately as:

```python
n.hit or (n.missed and n.y > HEIGHT + 10)
```

So a hit note gets removed immediately, while a missed note remains until it goes below the screen.

---

12. Current game-over condition

You initialize:

```python
self.misses = 0
```

Every automatic miss increments it:

Python Run

```python
self.misses += 1
```

And:

Python Run

```python
if self.misses >= 15:
    self.game_over = True
```

So:

15 misses → game over

The game doesn't currently have a separate summary screen. It simply overlays:

```
GAME OVER

Final Score: ...
Max Combo: ...x

Press R to Restart
```

---

13. Responsibility of each function

Here's the easiest way to remember your architecture for the viva:

Function/Class	Responsibility
`main.py`	Starts the game
`GameEngine.__init__()`	Initializes Pygame and game objects
`GameEngine.reset()`	Resets game state
`spawn_note()`	Creates a random note
`handle_events()`	Handles keyboard/window events
`process_tap()`	Finds a note and calculates hit/grade
`update()`	Moves notes, spawns notes, detects misses
`draw()`	Draws game visuals/HUD

Function/Class	Responsibility
`run()`	Main game loop
`Note.__init__()`	Creates a note
`Note.update()`	Moves the note
`Note.get_rect()`	Returns its drawing rectangle

---

14. Implementation plan for your four requirements

I would **not rewrite the game**. I'd make four targeted changes.

Feature 1 — Sound effects

Add three sound effects:

```
PERFECT → perfect.wav
GREAT   → great.wav
OK      → ok.wav
```

Load them once in `GameEngine.__init__()`:

Python Run

```python
self.sounds = {
    "PERFECT": pygame.mixer.Sound(...),
    "GREAT": pygame.mixer.Sound(...),
    "OK": pygame.mixer.Sound(...)
}
```

Then inside `process_tap()`:

```
calculate grade
      ↓
play corresponding sound
      ↓
update score/combo
```

We won't touch the actual hit-detection architecture.

Feature 2 — Hold notes

Extend `Note`.

Currently:

Python Run

```
Note
 ├── lane
 ├── y
 ├── speed
 ├── hit
 └── missed
```

We'll add something like:

```
is_hold
hold_duration
holding
hold_completed
```

A hold note will behave roughly like:

```
player presses key
       ↓
note reaches hit window
       ↓
start hold
       ↓
player keeps key pressed
       ↓
1 second
       ↓
successful hold
```

We'll probably use Pygame's:

Python Run

```
pygame.key.get_pressed()
```

for continuous key-state checking.

That is different from:

Python Run

```
KEYDOWN
```

because `KEYDOWN` only tells us that the key was initially pressed.

Feature 3 — BPM-synchronized spawning

This is the biggest architectural change, but still fairly small.

Instead of:

Python Run

```python
spawn_interval = 45
```

we introduce:

Python Run

```python
BPM = ...
```

The fundamental formula is:

```
seconds per beat = 60 / BPM
```

For example, at 120 BPM:

```
60 / 120 = 0.5 seconds/beat
```

So:

```
Beat 1 → 0.0 s
Beat 2 → 0.5 s
Beat 3 → 1.0 s
Beat 4 → 1.5 s
...
```

Rather than counting arbitrary frames, we'll maintain elapsed time / beat timing and spawn notes according to the beat.

This also makes the game much easier to explain in the viva:

> "The note generation is synchronized to a fixed BPM. The interval between beats is calculated as 60 divided by BPM."

Feature 4 — Grade summary

Add counters to `reset()`:

Python Run

```python
self.perfects = 0
self.greats = 0
self.oks = 0
```

Then when a grade happens:

```
PERFECT → perfects += 1
GREAT   → greats += 1
OK      → oks += 1
MISS    → misses += 1
```

At game over, calculate:

```
total attempts =
PERFECT + GREAT + OK + MISS
```

and accuracy, for example:

```
accuracy =
(300×PERFECT + 200×GREAT + 100×OK)
/
(300×total notes)
× 100
```

We can choose the exact accuracy formula based on what your assignment expects.

The game-over screen can then become:

```
GAME OVER

PERFECT : 12
GREAT   : 8
OK      : 5
MISS    : 3

Accuracy: 76.4%

Score:  ...
Max Combo: ...x

Press R to Restart
```

The important part for your assignment

I recommend implementing them in this order:

```
1. Sound effects
       ↓
2. Grade counters + summary
       ↓
3. BPM spawning
       ↓
4. Hold notes
```

This lets you make **small, understandable commits** instead of dumping one huge AI-generated change into GitHub.

And for your vibe-coding submission, we can also make the AI prompts progressively ask for **analysis** → **implementation** → **testing** → **debugging**, so your submitted ChatGPT history demonstrates that you understood and guided the development rather than simply asking for "write the whole project."

Yes. We can replace only the **spawn timing mechanism** and leave your note movement, scoring, hit detection, hold notes, and sounds untouched.

How the BPM timing works

We'll define:

Python Run

```
BPM = 120
```

The interval between beats is:

```
beat interval = 60 / BPM
              = 60 / 120
```

```
= 0.5 seconds
```

So the timing becomes:

```
0.0 s → Beat 1 → spawn
0.5 s → Beat 2 → spawn
1.0 s → Beat 3 → spawn
1.5 s → Beat 4 → spawn
2.0 s → Beat 5 → spawn
...
```

Instead of:

```
45 frames → spawn
```

we'll use elapsed time from `pygame.time.get_ticks()`.

This is preferable to counting frames because **BPM represents real time**, not the number of frames rendered.

---

1. Add the BPM constant

At the top of `game/game_engine.py`, alongside your existing constants:

Python Run

```python
WIDTH, HEIGHT = 480, 640
FPS = 60
HIT_Y = HEIGHT - 80
HIT_WINDOW = 30
BG = (15, 10, 25)
LANE_W = WIDTH // LANES

BPM = 120
BEAT_INTERVAL = 60 / BPM
```

Therefore:

Python Run

```python
BEAT_INTERVAL
```

will be:

```
0.5 seconds
```

2. Change the timing variables in `reset()`

You currently have:

Python Run

```python
self.spawn_timer = 0
self.spawn_interval = 45
self.speed = 5
self.frame = 0
```

Remove the old spawn timer variables and replace them with:

Python Run

```python
self.last_beat_time = pygame.time.get_ticks()
self.beat_count = 0
self.speed = 5
self.frame = 0
```

So that section becomes:

Python Run

```python
def reset(self):
    self.notes = []
    self.score = 0
    self.combo = 0
    self.max_combo = 0
    self.misses = 0

    self.last_beat_time = pygame.time.get_ticks()
    self.beat_count = 0

    self.speed = 5
    self.frame = 0
    self.feedback = []
    self.game_over = False
```

Why `last_beat_time` ?

It stores:

> "When did the previous beat happen?"

Then we compare the current time against it.

3. Replace the old spawning logic in `update()`

You currently have:

Python

```python
self.frame += 1
self.spawn_timer += 1

if self.spawn_timer >= self.spawn_interval:
    self.spawn_note()
    self.spawn_timer = 0

    if self.frame % 600 == 0:
        self.speed = min(10, self.speed + 0.5)
        self.spawn_interval = max(25, self.spawn_interval - 2)
```

Remove that entire spawning section.

Replace it with:

Python

```python
self.frame += 1

current_time = pygame.time.get_ticks()

if current_time - self.last_beat_time >= BEAT_INTERVAL * 1000:
    self.spawn_note()
    self.last_beat_time += int(BEAT_INTERVAL * 1000)
    self.beat_count += 1

    if self.frame % 600 == 0:
        self.speed = min(10, self.speed + 0.5)
```

---

4. One important improvement

Because your game runs at 60 FPS, we shouldn't rely on a single exact frame occurring at exactly 500 ms.

The condition:

Python

```python
current_time - self.last_beat_time >= BEAT_INTERVAL * 1000
```

means:

"Has at least one beat interval passed?"

So if the game happens to check at:

```
499 ms → nothing
501 ms → spawn
```

that's fine.

The next beat remains scheduled relative to the previous beat because of:

```python
self.last_beat_time += int(BEAT_INTERVAL * 1000)
```

rather than:

```python
self.last_beat_time = current_time
```

This prevents small frame-timing delays from accumulating over the entire game.

---

5. What happened to the old speed adjustment?

Your old code had:

```python
if self.frame % 600 == 0:
    self.speed = min(10, self.speed + 0.5)
    self.spawn_interval = max(25, self.spawn_interval - 2)
```

We **keep the speed adjustment**:

```python
if self.frame % 600 == 0:
    self.speed = min(10, self.speed + 0.5)
```

But we remove:

```python
self.spawn_interval = max(25, self.spawn_interval - 2)
```

because `spawn_interval` no longer exists.

This is important:

> **Note movement speed can still increase, but the BPM stays fixed at 120.**

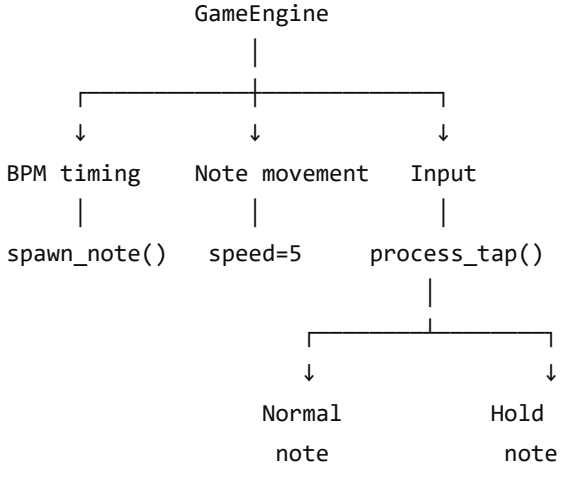
So the game becomes faster because notes travel faster, **not because beats become faster than 120 BPM**.

Final timing flow

Your game will now work like this:

```
Game starts
    ↓
BPM = 120
    ↓
60 / 120
    ↓
0.5 seconds per beat
    ↓
pygame.time.get_ticks()
    ↓
Has 0.5 seconds passed?
    ↓
YES
    ↓
spawn_note()
    ↓
schedule next beat
    ↓
repeat
```

Your existing systems remain separate:



```
      |                 |
grading           hold timing
      |                 |
sound effects     completion
```

So **BPM synchronization only changes when `spawn_note()` gets called**. It does not interfere with your existing scoring, hit detection, hold-note state, or sound effects.

One thing to test afterward

At 120 BPM, you should see approximately **one new note every 0.5 seconds**, i.e. about **120 note spawns per minute**.

Because your notes can now be either normal or hold notes, that's **120 note objects per minute**, not necessarily 120 normal notes.

I implemented BPM-synchronized spawning. Help me verify that it is actually synchronized to 120 BPM rather than just appearing to work visually.

Explain:

- how 120 BPM becomes a 0.5 second beat interval,
- how elapsed time is converted into a beat number,
- why using elapsed time is preferable to relying only on frame count,
- what happens if the game experiences a frame delay.

Show more

Yes. The important distinction is that your game is now using **real elapsed time** to decide when a beat occurs, rather than assuming that a certain number of frames equals a beat.

1. How 120 BPM becomes 0.5 seconds

BPM means **beats per minute**.

Your code has:

Python Run

```python
BPM = 120
```

The formula is:

```
beat interval = 60 / BPM
```

Therefore:

```
60 / 120 = 0.5 seconds
```

So the expected beat times are approximately:

```
Beat 1 → 0.0 s
Beat 2 → 0.5 s
Beat 3 → 1.0 s
Beat 4 → 1.5 s
Beat 5 → 2.0 s
Beat 6 → 2.5 s
...
```

That means you expect **2 beats per second**, or **120 beats per minute**.

---

2. How elapsed time becomes a beat number

Your implementation keeps track of when the previous beat occurred:

Python Run

```python
self.last_beat_time = pygame.time.get_ticks()
```

Then every update:

Python Run

```python
current_time = pygame.time.get_ticks()
```

and checks:

Python Run

```python
if current_time - self.last_beat_time >= BEAT_INTERVAL * 1000:
```

Since:

Python Run

```
BEAT_INTERVAL = 0.5
```

the condition becomes approximately:

Python Run

```
if elapsed_time >= 500:
```

because `pygame.time.get_ticks()` is measured in milliseconds.

When the condition is satisfied:

Python Run

```
self.spawn_note()
self.last_beat_time += int(BEAT_INTERVAL * 1000)
self.beat_count += 1
```

So `beat_count` represents the number of beats that have been reached.

For example:

```
Elapsed time       Beat count

~0.5 sec              1
~1.0 sec              2
~1.5 sec              3
~2.0 sec              4
~2.5 sec              5
```

One important clarification

Your current implementation doesn't calculate the beat number directly using:

Python Run

```
beat_number = elapsed_time / beat_interval
```

Instead, it **advances `beat_count` by one whenever another 0.5-second interval has elapsed**.

Conceptually, they're equivalent:

```
beat number ≈ elapsed time / 0.5
```

but your implementation uses the accumulated `last_beat_time` approach.

3. Why elapsed time is better than frame counting

Your game runs at:

Python Run

```python
FPS = 60
```

A frame-based system might say:

```
30 frames = 1 beat
```

because:

```
30 frames / 60 FPS = 0.5 seconds
```

The problem is that **FPS isn't guaranteed to be perfectly constant**.

Imagine a temporary slowdown:

```
Frame 1
Frame 2
Frame 3
...
game gets busy
...
Frame takes longer
...
```

If you simply counted frames, you would assume:

```
30 frames = 0.5 seconds
```

even if those 30 frames actually took 0.55 or 0.60 seconds.

Elapsed-time timing instead asks:

> "Has 500 milliseconds actually passed?"

That is much closer to what BPM means.

---

4. What happens if there is a frame delay?

Suppose a beat is supposed to occur at:

```
1.500 seconds
```

but the game experiences a temporary delay.

Maybe the update loop checks at:

```
1.48 s → not enough time
1.56 s → enough time
```

The game will spawn the note at the next update after the 500 ms threshold is crossed.

So there may be a tiny delay in **when the game notices the beat**, but it doesn't redefine the BPM.

More importantly, your code does:

Python Run

```python
self.last_beat_time += int(BEAT_INTERVAL * 1000)
```

rather than:

Python Run

```python
self.last_beat_time = current_time
```

That's useful because you're maintaining the original beat schedule.

For example:

```
Expected:
0.5
1.0
1.5
2.0
2.5
```

If one update happens late:

```
Expected beat: 1.5
Actually detected: 1.56
```

you don't say:

> "Okay, the new beat is 1.56, so the next one is 2.06."

Instead, the schedule continues toward:

```
1.5
2.0
2.5
```

This prevents timing drift from accumulating.

Practical test: verify 120 BPM

Don't rely on whether the notes *look* evenly spaced. We can test the actual timing.

Step 1 — Temporarily display the beat information

For testing, add this temporarily inside `draw()`:

Python Run

```python
beat_text = self.font.render(
    f"Beat: {self.beat_count}",
    True,
    (220, 220, 220)
)
self.screen.blit(beat_text, (10, 70))
```

Now you'll see:

```
Score: ...
Combo: ...
Beat: 17
```

Step 2 — Run the game

Bash

```bash
python main.py
```

Watch the `Beat:` value.

It should increase approximately:

```
0
1
2
3
4
```

```
5
...
```

with roughly **0.5 seconds between increments**.

Step 3 — Do a longer timing test

Instead of trying to judge 0.5 seconds by eye, count **20 beats**.

At 120 BPM:

```
20 beats × 0.5 seconds
= 10 seconds
```

So:

> From the first beat to the twentieth beat should be approximately 10 seconds.

Because the first beat occurs after the initial timing interval in your current implementation, use the difference between two known beat transitions rather than the moment the game window opens.

For example:

```
Beat 5 timestamp
        ↓
Beat 25 timestamp
        ↓
Expected difference ≈ 10 seconds
```

Even better: temporary terminal verification

For a very clear verification, temporarily add:

Python

```python
print(
    f"Beat {self.beat_count} "
    f"at {current_time / 1000:.3f}s"
)
```

immediately after:

Python Run

```
self.beat_count += 1
```

Then run:

Bash

```
python main.py
```

You should see something approximately like:

```
Beat 1 at 0.502s
Beat 2 at 1.001s
Beat 3 at 1.503s
Beat 4 at 2.002s
Beat 5 at 2.501s
Beat 6 at 3.004s
```

The exact numbers won't be identical because the game loop doesn't execute at infinitely precise intervals.

But the differences should be close to:

```
~0.500 seconds
```

For example:

```
1.001 - 0.502 = 0.499
1.503 - 1.001 = 0.502
2.002 - 1.503 = 0.499
```

That's strong evidence that the spawning mechanism is actually tied to **120 BPM**.

After testing, remove the temporary `print()` and/or `Beat:` display so you don't change the final gameplay/UI.

implement **Requirement 4: Grade Summary Screen**.

When the game ends, I want a summary showing:

- PERFECT count

- GREAT count
- OK count
- MISS count
- accuracy percentage
- final score

Show more

Yes. We can add this cleanly to your existing `GameEngine` without changing the gameplay loop or the other three requirements.

The main idea is:

```
reset()
  ↓
initialize counters
  ↓
successful normal note → increment PERFECT/GREAT/OK
successful hold → increment PERFECT
missed normal/hold → increment MISS
  ↓
game_over = True
  ↓
draw() displays summary
  ↓
R → reset()
```

1. Initialize the counters

In `reset()`, alongside your existing score/combo/miss variables, add:

Python Run

```python
self.perfects = 0
self.greats = 0
self.oks = 0
self.misses = 0
```

You already have:

Python Run

```python
self.misses = 0
```

so **don't add it twice**. Your section should look like:

Python Run

```
self.score = 0
self.combo = 0
self.max_combo = 0

self.perfects = 0
self.greats = 0
self.oks = 0
self.misses = 0
```

This ensures restarting the game also resets the summary.

---

2. Increment the normal-note grade counters

In `process_tap()`, you already have the three grading branches.

PERFECT

Change:

Python Run

```
if best_dist < 8:
    grade, pts = "PERFECT", 300
    col = (255, 220, 0)
    self.perfect_sound.play()
```

to:

Python Run

```
if best_dist < 8:
    grade, pts = "PERFECT", 300
    col = (255, 220, 0)
    self.perfects += 1
    self.perfect_sound.play()
```

GREAT

Change:

Python Run

```
elif best_dist < 18:
    grade, pts = "GREAT", 200
```

```
    col = (100, 220, 100)
    self.great_sound.play()
```

to:

Python Run

```
elif best_dist < 18:
    grade, pts = "GREAT", 200
    col = (100, 220, 100)
    self.greats += 1
    self.great_sound.play()
```

OK

Change:

Python Run

```
else:
    grade, pts = "OK", 100
    col = (180, 180, 255)
    self.ok_sound.play()
```

to:

Python Run

```
else:
    grade, pts = "OK", 100
    col = (180, 180, 255)
    self.oks += 1
    self.ok_sound.play()
```

So every successfully hit **normal note** contributes exactly one grade.

---

3. Count normal-note misses

You have two kinds of misses.

Player presses incorrectly

In `process_tap()`, you currently have:

Python Run

```
else:
    self.combo = 0
```

```
    self.feedback.append([
        "MISS",
        (220,60,60),
        40,
        lane_x,
        HIT_Y - 30
    ])
```

Add:

Python Run

```
self.misses += 1
```

So:

Python Run

```
else:
    self.misses += 1
    self.combo = 0
    self.feedback.append([
        "MISS",
        (220,60,60),
        40,
        lane_x,
        HIT_Y - 30
    ])
```

This ensures an incorrect tap is represented in the summary.

---

4. Count automatically missed notes

Your `update()` already handles notes that pass the hit area:

Python Run

```
note.missed = True
self.misses += 1
self.combo = 0
```

So **don't add another increment there**.

This already correctly counts an unhit normal note as a MISS.

---

5. Count successful hold notes

In your hold completion code, you currently have something like:

Python Run

```python
elif pygame.time.get_ticks() - note.hold_start >= note.hold_duration * 1000:
    note.holding = False
    note.hold_complete = True
    note.hit = True

    self.combo += 1
    self.max_combo = max(self.max_combo, self.combo)
    self.score += 300 * max(1, self.combo // 5)

    self.feedback.append([
        "PERFECT",
        (255, 220, 0),
        40,
        note.lane * LANE_W + LANE_W // 2,
        HIT_Y - 30
    ])
```

Add:

Python Run

```python
self.perfects += 1
```

So it becomes:

Python Run

```python
elif pygame.time.get_ticks() - note.hold_start >= note.hold_duration * 1000:
    note.holding = False
    note.hold_complete = True
    note.hit = True

    self.perfects += 1

    self.combo += 1
    self.max_combo = max(self.max_combo, self.combo)
    self.score += 300 * max(1, self.combo // 5)

    self.feedback.append([
        "PERFECT",
        (255, 220, 0),
        40,
        note.lane * LANE_W + LANE_W // 2,
```

```
        HIT_Y - 30
    ])
```

A completed hold therefore counts as:

```
PERFECT +1
```

which matches your current gameplay behavior where a completed hold gives the PERFECT-style 300 points.

---

6. Count early-release hold misses

You already have:

Python Run

```python
if not keys[key]:
    note.holding = False
    note.missed = True
    self.misses += 1
    self.combo = 0
```

That's already correct.

Do not add another `self.misses += 1`.

An early-release hold therefore contributes:

```
MISS +1
```

---

7. Accuracy formula

You requested:

```
PERFECT = 100%
GREAT   = 75%
OK      = 50%
MISS    = 0%
```

So we calculate weighted accuracy.

First calculate total judged notes:

Python Run

```python
total_notes = (
    self.perfects
    + self.greats
    + self.oks
    + self.misses
)
```

Then:

Python Run

```python
accuracy = (
    self.perfects * 100
    + self.greats * 75
    + self.oks * 50
) / total_notes
```

If there are no judged notes:

Python Run

```python
accuracy = 0
```

So use:

Python Run

```python
if total_notes > 0:
    accuracy = (
        self.perfects * 100
        + self.greats * 75
        + self.oks * 50
    ) / total_notes
else:
    accuracy = 0
```

Example

Suppose:

```
PERFECT = 5
GREAT   = 3
OK      = 1
MISS    = 1
```

Then:

```
Total = 10

Accuracy =
(5×100 + 3×75 + 1×50 + 1×0) / 10

= (500 + 225 + 50) / 10

= 77.5%
```

---

8. Replace the existing game-over display

You currently have a game-over section in `draw()` that displays:

```
GAME OVER

Final Score: ...
Max Combo: ...

Press R to Restart
```

Replace that entire section:

Python Run

```python
if self.game_over:
    ...
```

with:

Python Run

```python
if self.game_over:
    ov = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    ov.fill((0, 0, 0, 180))
    self.screen.blit(ov, (0, 0))

    total_notes = (
        self.perfects
        + self.greats
        + self.oks
        + self.misses
    )

    if total_notes > 0:
        accuracy = (
            self.perfects * 100
            + self.greats * 75
```

```
            + self.oks * 50
        ) / total_notes
    else:
        accuracy = 0

    title = self.big_font.render(
        "GAME OVER",
        True,
        (220, 60, 60)
    )

    perfect = self.font.render(
        f"PERFECT: {self.perfects}",
        True,
        (255, 220, 0)
    )

    great = self.font.render(
        f"GREAT:   {self.greats}",
        True,
        (100, 220, 100)
    )

    ok = self.font.render(
        f"OK:      {self.oks}",
        True,
        (180, 180, 255)
    )

    miss = self.font.render(
        f"MISS:    {self.misses}",
        True,
        (220, 60, 60)
    )

    accuracy_text = self.font.render(
        f"Accuracy: {accuracy:.1f}%",
        True,
        (220, 220, 220)
    )

    score_text = self.font.render(
        f"Final Score: {self.score}",
        True,
        (220, 220, 220)
    )

    combo_text = self.font.render(
        f"Max Combo: {self.max_combo}x",
        True,
        (255, 220, 80)
    )
```

```
        restart = self.font.render(
            "Press R to Restart",
            True,
            (160, 160, 160)
        )

        self.screen.blit(
            title,
            (WIDTH // 2 - title.get_width() // 2, 80)
        )

        self.screen.blit(
            perfect,
            (WIDTH // 2 - perfect.get_width() // 2, 160)
        )

        self.screen.blit(
            great,
            (WIDTH // 2 - great.get_width() // 2, 200)
        )

        self.screen.blit(
            ok,
            (WIDTH // 2 - ok.get_width() // 2, 240)
        )

        self.screen.blit(
            miss,
            (WIDTH // 2 - miss.get_width() // 2, 280)
        )

        self.screen.blit(
            accuracy_text,
            (WIDTH // 2 - accuracy_text.get_width() // 2, 340)
        )

        self.screen.blit(
            score_text,
            (WIDTH // 2 - score_text.get_width() // 2, 390)
        )

        self.screen.blit(
            combo_text,
            (WIDTH // 2 - combo_text.get_width() // 2, 430)
        )

        self.screen.blit(
            restart,
            (WIDTH // 2 - restart.get_width() // 2, 510)
        )
```

What the final screen will look like

Approximately:

```
          GAME OVER

        PERFECT: 12
        GREAT:    8
        OK:       5
        MISS:     3

        Accuracy: 74.6%

        Final Score: 5600
        Max Combo: 9x

      Press R to Restart
```

The existing transparent dark overlay is preserved.

How all four result types are now counted

Your complete flow becomes:

```
NORMAL NOTE
     │
     ├── PERFECT → perfects += 1
     │
     ├── GREAT   → greats += 1
     │
     └── OK      → oks += 1
```

For misses:

```
NORMAL NOTE
     │
     ├── bad tap
     │      ↓
     │   misses += 1
     │
     └── passes hit zone
            ↓
         misses += 1
```

For hold notes:

```
HOLD NOTE
     │
     ├── held 1 second
     │       ↓
     │   perfects += 1
     │
     └── released early
           ↓
         misses += 1
```

So **hold completions and early-release misses are included in exactly the same summary counters as normal notes**.

Viva explanation

If they ask **"How did you calculate accuracy?"**, say:

> "I maintain separate counters for PERFECT, GREAT, OK, and MISS. Each result contributes a different weight: PERFECT contributes 100%, GREAT 75%, OK 50%, and MISS 0%. I calculate the weighted sum and divide it by the total number of judged notes. Hold notes are also included: a completed hold is counted as PERFECT, while an early release is counted as MISS."

And importantly, the summary is **not a separate game mode**. It's simply another display state controlled by your existing:

Python Run

```python
self.game_over
```

flag, so the existing game architecture stays intact.

ChatGPT can make mistakes. Check important info.